

NutriChat — Evolución del asistente (V1 → V2 → V3)

Para alguien que quiere entender cómo funciona un chatbot con IA, qué decisiones se tomaron y por qué.

Índice

- [1. Qué es NutriChat y qué puede hacer](#)
 - [2. Cómo funciona un chatbot con IA \(conceptos base\)](#)
 - [3. Diagrama: flujo completo de una consulta](#)
 - [4. V1 — Asistente básico con herramientas](#)
 - [5. V2 — Asistente más analítico](#)
 - [6. V3 — Asistente con base de conocimiento propia](#)
 - [7. Resumen de archivos modificados](#)
-

1. Qué es NutriChat y qué puede hacer

NutriChat es el asistente de IA dentro de la app de nutrición. Responde preguntas sobre alimentación usando dos fuentes:

- **Su conocimiento general** (entrenado con millones de textos sobre nutrición)
- **Tus datos reales** de la app: perfil, objetivo, comidas registradas, recetas favoritas

Lo que NO puede hacer (por diseño):

- Hablar de temas que no sean nutrición/alimentación
 - Diagnosticar enfermedades
 - Reemplazar a un nutricionista o médico
-

2. Cómo funciona un chatbot con IA (conceptos base)

Antes de ver los cambios, es importante entender algunos conceptos.

El modelo de IA (LLM)

Un **LLM** (*Large Language Model*) como Gemini es una IA que aprendió a escribir texto respondiendo preguntas. No "piensa" — predice qué texto tiene sentido dado lo que recibió. Cuanto más contexto le das, mejor responde.

El System Prompt

Es la instrucción que define quién es el asistente. Se envía una sola vez antes de la conversación. Por ejemplo:

"Sos NutriChat. Solo respondés sobre nutrición. No diagnosticás enfermedades..."

Conversación multi-turn

Cuando hablás con alguien, recordás lo que dijiste antes. Un LLM no tiene memoria propia — cada vez que le mandás un mensaje, le tenés que mandar también todo lo anterior. Eso se llama **multi-turn**: enviar el historial completo de la conversación junto con el mensaje nuevo.

Tools (herramientas)

Las tools son funciones que el asistente puede "activar" para buscar información real. Sin tools, Gemini solo puede responder con lo que sabe de su entrenamiento. Con tools, puede buscar en la base de datos: tus comidas de hoy, tu perfil, tus recetas.

Context Injection

Es el proceso de adjuntar datos reales al mensaje antes de enviarlo a la IA. En lugar de que la IA invente respuestas, le pasamos los datos concretos y ella los interpreta:

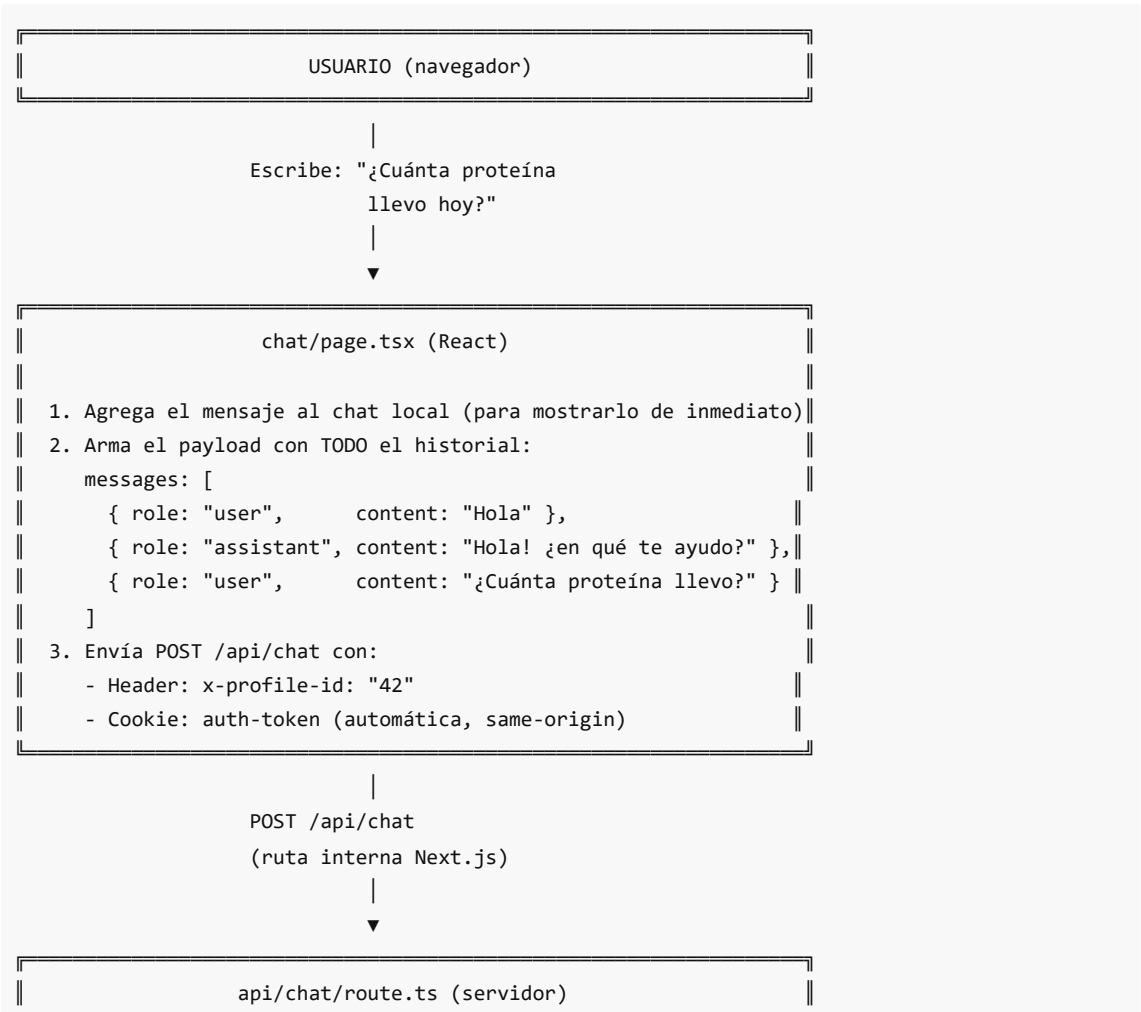
```
[Datos del usuario]
Perfil: Matías | Objetivo: Bajar grasa
Comidas hoy: Desayuno 350 kcal, Almuerzo 600 kcal
Total: 950 kcal | 72g proteína

[Pregunta]
¿Qué me conviene cenar?
```

Gemini recibe todo esto junto y puede dar una respuesta personalizada.

3. Diagrama: flujo completo de una consulta

Este diagrama muestra exactamente qué pasa desde que el usuario escribe algo hasta que recibe la respuesta.



PASO 1 – Validación

```
¿Tiene auth-token? → No → 401 Unauthorized|
¿Tiene x-profile-id? → No → 400 Bad Request|
¿Mensaje vacío? → No → 400 Bad Request|
```

PASO 2 – ¿Es sobre nutrición?

```
isNutritionRelated("¿Cuánta proteína...?")|
→ Detecta "proteína" en la lista de palabras|
→ SÍ → continúa|
→ NO → responde con mensaje de rechazo,|
      sin llamar a Gemini
```

PASO 3 – Recolectar datos reales (tools)

```
SIEMPRE: getActiveProfileTool()|
→ GET http://localhost:4000/profiles|
→ Resultado: "Matías | Objetivo: bajar|
              grasa | Hoy: lunes 13 de abril"|

SI pregunta por macros/hoy:|
  wantsPersonalFoodData() → SÍ|
  → getTodayMealsTool()|
  → getNutritionSummaryTool()|
  → GET http://localhost:4000/meals|
  → Resultado: "Desayuno 350 kcal 30g prot"|

SI pregunta por la semana:|
  wantsWeeklyProgress() → NO (esta vez)|

SI pregunta por recetas:|
  wantsRecipes() → NO (esta vez)|
```

PASO 4 – Ensamblar contexto

```
contextBlocks = [|
  "Perfil activo: Matías|
  Objetivo: bajar grasa|
  Fecha de hoy: lunes 13 de abril 2026",|
|
  "Comidas registradas hoy (2026-04-13):|
  • Desayuno: 350 kcal | 30g prot...|
  Totales: 350 kcal | 30g proteína..."|
|
]
```

PASO 5 – Construir conversación multi-turn para Gemini

```
contents = [
  { role: "user", text: "Hola" },
  { role: "model", text: "Hola! ¿en qué..."},
  { role: "user", text:
    "[Datos actuales del usuario]
    Perfil activo: Matías...
    Comidas de hoy: Desayuno 350 kcal...
    [Consulta del usuario]
    ¿Cuánta proteína llevo hoy?" }
]
```

POST a API de Google (Gemini)
+ system prompt separado:
"Sos NutriChat. Solo nutrición..."

GEMINI 2.5 FLASH (Google)

Recibe:

- Instrucción del sistema: quién es, qué puede/no puede hacer
- Historial completo de la conversación
- Datos reales del usuario pegados al último mensaje

Procesa todo junto y genera una respuesta natural:

"Hoy llevás 30g de proteína con el desayuno. Para bajar
grasa te recomiendo apuntar a 120-140g diarios..."

{ reply: "Hoy llevás 30g..." }

chat/page.tsx (React)

Muestra la respuesta como burbuja del asistente en el chat

USUARIO ve la respuesta

Por qué el contexto va pegado al último mensaje y no como mensaje aparte

Gemini exige que los turnos se alternen estrictamente: user → model → user → model . No se puede insertar un mensaje de "sistema" en el medio de la conversación. Por eso los datos se pegan dentro del último mensaje del usuario, usando un formato claro:

```
[Datos actuales del usuario en la app]
Perfil activo: Matías
```

```
Objetivo: bajar grasa
...

[Consulta del usuario]
¿Cuánta proteína llevo hoy?
```

Gemini lee esto como un solo mensaje y sabe distinguir qué son datos de contexto y qué es la pregunta real.

4. V1 — Asistente básico con herramientas

Objetivo

Que el asistente funcione de forma fluida y correcta con:

- Historial de conversación real (multi-turn)
 - Perfil del usuario siempre disponible
 - Comidas del día y macros cuando se pregunta
 - Recetas favoritas cuando se pregunta
 - Sin respuestas genéricas ni robóticas
-

Qué había antes y qué estaba mal

El historial no se enviaba al servidor

Cada mensaje se enviaba solo:

```
{ "message": "¿cuánta proteína necesito?" }
```

Para Gemini, cada pregunta era una conversación nueva. Si antes preguntaste algo, no lo recordaba.

El perfil no siempre estaba disponible

El perfil (nombre + objetivo) solo se buscaba si el usuario escribía frases exactas como "*mi perfil*" o "*mi objetivo*". Si preguntaba "*¿qué desayuno?*", Gemini respondía sin saber si esa persona quería bajar grasa o ganar músculo.

Respuestas hardcodeadas (sin IA)

Para algunas preguntas, el código devolvía respuestas armadas a mano sin pasar por Gemini:

```
"Hoy llevás 2 comidas registradas: Comida desayuno, Comida almuerzo.
Acumulás 950 kcal, 72g de proteína..."
```

Era un texto fijo generado por código. Robótico, sin análisis, sin sugerencias.

El error 404 — bug crítico

El chat usaba `apiFetch("/api/chat")`. Esta función está diseñada para llamar al servidor de Express (el backend), pero `/api/chat` es una ruta interna de Next.js, no de Express. El resultado era un 404 porque Express no tiene esa ruta.

¿Por qué pasó? `apiFetch` agrega `/api/proxy` delante de todo:

- Entrada: `/api/chat`

- Lo convierte en: `/api/proxy/api/chat`
- El proxy reenvía a: `http://localhost:4000/api/chat`
- Express: ruta no encontrada → **404**

Qué se cambió en V1

Opción A: Context Injection manual (elegida)

Buscar los datos antes de llamar a la IA y pegarlos al mensaje.

Ventaja	Desventaja
Simple de implementar	Siempre busca datos aunque no hagan falta
Control total sobre qué datos se pasan	El contexto puede crecer con muchos datos
No requiere capacidades extra del modelo	Hay que mantener la lógica de detección de intent

Opción B: Function Calling nativo de Gemini

Gemini decide qué tools llamar él solo.

Ventaja	Desventaja
La IA decide qué necesita	Más complejo de implementar
Más flexible	Gemini puede llamar tools innecesarias
Menos código de detección manual	Requiere definir schema de funciones

¿Por qué elegimos la Opción A? Es más predecible, más fácil de debuggear y adecuada para V1. La lógica de "cuándo buscar datos" es simple y controlable. En una fase futura se puede migrar a Function Calling.

Cambios implementados en V1

1. `chat/page.tsx` — Enviar historial completo

```
// Antes: solo el mensaje actual
body: JSON.stringify({ message: trimmed })

// Ahora: historial completo + mensaje nuevo
body: JSON.stringify({
  messages: [...historialAnterior, { role: "user", content: trimmed }]
})
```

2. `chat/page.tsx` — Usar `fetch` directo en vez de `apiFetch`

```
// Antes (rompía con 404):
const res = await apiFetch("/api/chat", { ... })

// Ahora (correcto):
const res = await fetch("/api/chat", {
  headers: { "Content-Type": "application/json", "x-profile-id": profileId },
```

```
...
})
```

La cookie de autenticación se envía automáticamente en peticiones same-origin.

3. route.ts — Perfil siempre incluido

Para toda consulta de nutrición, el perfil se carga automáticamente. Gemini siempre sabe con quién habla y cuál es su objetivo.

4. route.ts — Multi-turn real para Gemini

```
// Antes: todo en un string gigante
contents: "Sistema: ...\nContexto: ...\nUsuario: ...\nAsistente: ..."

// Ahora: array de turnos con roles
contents: [
  { role: "user", parts: [{ text: "mensaje 1" }] },
  { role: "model", parts: [{ text: "respuesta 1" }] },
  { role: "user", parts: [{ text: "[Datos]\n...\n[Pregunta]\nmensaje 2" }] },
]
// El system prompt va separado en config.systemInstruction
```

5. route.ts — Eliminar respuestas hardcodeadas

Los datos se pasan como contexto y Gemini siempre genera la respuesta. Resultado: respuestas naturales, con análisis y sugerencias.

6. system-prompt.ts — Prompt mejorado

Se agregaron instrucciones para que Gemini use los datos de la app de forma natural (no los repita como lista cruda), adapte el tono al objetivo del perfil, y use párrafos cortos aptos para mobile.

5. V2 — Asistente más analítico

Objetivo

Que el asistente pueda analizar no solo el día actual, sino la semana completa: consistencia, promedios, días con y sin registro.

Qué se agregó

Nueva tool: `getWeeklySummaryTool`

Busca todas las comidas del perfil y las agrupa por día para los últimos 7 días. Calcula:

- Calorías, proteína, carbs y grasas por día
- Días con y sin registro
- Promedios de la semana (sobre días con datos)

El resultado que recibe Gemini se ve así:

```
Resumen de los últimos 7 días (2026-04-06 al 2026-04-13):
• lunes 2026-04-06: sin registro
```

- martes 2026-04-07: 1840 kcal | 95g prot | 210g carbs | 65g grasas (3 comidas)
- miércoles 2026-04-08: sin registro
- jueves 2026-04-09: 1620 kcal | 88g prot | 190g carbs | 55g grasas (3 comidas)
- viernes 2026-04-10: 2100 kcal | 110g prot | 240g carbs | 72g grasas (4 comidas)
- sábado 2026-04-11: sin registro
- domingo 2026-04-12: 1750 kcal | 92g prot | 200g carbs | 60g grasas (3 comidas)

Días con registro: 4 de 7

Promedios: 1827 kcal | 96g proteína | 210g carbohidratos | 63g grasas

Con estos datos, Gemini puede responder preguntas como:

- "¿Cómo voy en la semana?"
- "¿Tengo consistencia?"
- "¿Cuál fue mi promedio de calorías esta semana?"

¿Por qué no calculamos metas de calorías exactas?

Para calcular cuántas calorías necesita una persona se necesita: peso, altura, edad, nivel de actividad física. La app no tiene esos datos (aún). Sin ellos, cualquier número sería una suposición.

Lo que sí tenemos es el **objetivo del perfil** (bajar grasa, ganar músculo, etc.) y los **datos reales** de lo que comió. Gemini puede comentar las tendencias y comparar con lo que es razonable según el objetivo, sin inventar números exactos.

Detección de intent para V2

Se agregó `wantsWeeklyProgress()` que detecta palabras como: *semana, progreso, tendencia, consistencia, promedio, últimos días*.

Cuando se activa, se llama a `getWeeklySummaryTool` y el resultado se agrega al contexto de Gemini.

6. V3 — Asistente con base de conocimiento propia

¿Por qué no se implementó todavía?

V3 no se implementó porque **no existe el contenido que debería buscar**.

RAG (la técnica de V3) funciona así: guardas documentos propios (artículos, guías, tablas), y cuando el usuario pregunta algo, el sistema busca qué fragmentos son relevantes y se los pasa a la IA. Pero si no hay documentos propios, no hay nada que buscar.

Construir la infraestructura de búsqueda antes de tener contenido sería como instalar un buscador en una biblioteca vacía. No agrega valor y agrega complejidad.

La regla práctica es: primero el contenido, después la búsqueda.

¿Qué debería existir antes de implementar V3?

- Guías de nutrición propias de la app (por ejemplo: "cómo distribuir macros para bajar grasa")
- Tablas de alimentos con sus valores nutricionales
- Planes de comida tipo (desayunos, almuerzos, cenas saludables)
- Artículos curados sobre hábitos alimentarios

Una vez que exista ese contenido, V3 agrega mucho valor: el asistente puede citar información específica y confiable en lugar de depender solo de su entrenamiento general.

Objetivo (para cuando esté listo)

Que el asistente pueda responder usando documentos propios: guías nutricionales, artículos, tablas de macros por alimento.

¿Qué es RAG?

RAG (*Retrieval Augmented Generation*) es una técnica donde:

1. Los documentos se dividen en fragmentos y se guardan en una base de datos vectorial
2. Cuando el usuario pregunta algo, se busca qué fragmentos son más relevantes para esa pregunta
3. Esos fragmentos se adjuntan al mensaje (context injection)
4. La IA responde usando esa información específica

Opciones para implementarlo

Opción A: Base de datos vectorial externa (Pinecone, Weaviate)

Ventaja	Desventaja
Escala a millones de documentos	Servicio externo pago
Búsqueda semántica muy precisa	Más infraestructura para mantener

Opción B: pgvector (extensión de PostgreSQL)

Ventaja	Desventaja
Se integra en la misma base de datos	Limitado a escala mediana
Sin servicios externos	Requiere migración del schema

Opción C: Búsqueda en texto plano (simple, para empezar)

Ventaja	Desventaja
Zero infraestructura extra	No entiende sinónimos ni contexto
Implementación en horas	Resultados menos precisos

Recomendación para V3: Empezar con archivos Markdown simples (guías, tablas) y búsqueda por palabras clave. Si se necesita más, migrar a pgvector.

¿Cuándo tiene sentido?

Cuando la app tenga contenido propio: guías de nutrición personalizadas, planes de comida, artículos curados. Sin contenido propio, RAG no agrega valor.

7. Resumen de archivos modificados

Archivo	Qué cambió
---------	------------

<code>app/(app)/chat/page.tsx</code>	Envía historial completo; usa <code>fetch</code> directo (fix 404)
<code>app/api/chat/route.ts</code>	Multi-turn real; perfil siempre incluido; sin hardcoding; detección V2
<code>lib/assistant/system-prompt.ts</code>	Instrucciones mejoradas para usar datos reales y adaptar tono
<code>lib/assistant/tool-impl.ts</code>	Nueva <code>getWeeklySummaryTool</code> para resumen de 7 días
<code>lib/assistant/types.ts</code>	Nuevos tipos <code>WeeklyDaySummary</code> y <code>WeeklySummaryToolResult</code>